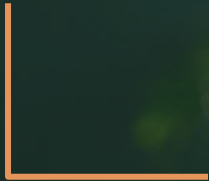


PROMETEO

Manejo de Excepciones



Índice

01

Excepciones

¿Qué es una excepción?

Tratamiento de excepciones

Sintaxis básica

02

Jerarquía de excepciones

03

Tipos de excepciones

Checked

Unchecked

Error

04

Excepciones personalizadas

throw / throws

A group of people are working in a modern office setting. In the foreground, a man with grey hair and a beard is smiling and looking back over his shoulder while working on a laptop. Behind him, several other people are also working on laptops, some looking towards the camera and others focused on their work. The office has a warm, casual atmosphere with wooden desks and modern decor.

1

Excepciones

PROMETEO

01

Introducción

Una **excepción** es un **evento anómalo** que puede ocurrir en tiempo de ejecución y que **altera el flujo normal** de ejecución.

Estos eventos ocurren de forma inesperada y deben ser tratados adecuadamente para garantizar la correcta ejecución del programa (evitar que finalice de forma abrupta).

No se deben utilizar excepciones para controlar el flujo normal del programa ya que están diseñadas para situaciones anómalas y no para validaciones comunes.

01

Introducción

Cuando se da una excepción se podrá: **capturar y manejar** o evitarla realizando cambios en la lógica del programa.

Java define una excepción como una **instancia** de la clase **Throwable** o de alguna de sus subclases. Es decir, como un objeto que representa un error o estado excepcional.

Introducción

El **tratamiento de excepciones** es un mecanismo fundamental para construir programas robustos y fiables:

- ✓ **Se manejan los errores de forma estructurada:** se separa el código para el tratamiento de las excepciones del código funcional.
- ✓ **Mejora de la robustez y confiabilidad** : al prever las posibles situaciones excepcionales y manejarlas adecuadamente, se consigue código más robusto.
- ✓ Se **informa al usuario** de forma personalizada cuando se producen las excepciones.
- ✓ **Flexibilidad:** se puede adaptar el comportamiento de la aplicación a las diferentes circunstancias.

01

Sintaxis básica

Java proporciona una **jerarquía de clases de excepciones** que en combinación con las sentencias try, catch, finally, throw y throws facilitan la gestión de las excepciones

- **try:** Define un bloque de código donde puede ocurrir una excepción.
- **catch:** Captura y maneja la excepción si ocurre en el bloque try. Se indica el nombre de la excepción que se quiere capturar. Si no se conoce, se utiliza la clase Exception (padre)
- **finally:** Contiene código que se ejecuta siempre, haya ocurrido una excepción o no. Se suele utilizar para cerrar ficheros, conexiones, liberar recursos, etc...
- **throw:** Lanza (crea una instancia) una excepción manualmente.
- **throws:** Indica que un método puede lanzar excepciones.

01

Sintaxis básica

Sintaxis básica:

```
try{  
    // instrucciones que pueden provocar una excepción  
}  
catch(NombreDeLaExcepcion instancia){  
    // código que maneja la excepción en caso de producirse  
}  
finally{  
    // Este bloque se ejecuta siempre independientemente del  
    // bloque del try o del catch  
}
```


01

Sintáxis básica

Se pueden añadir tantos `catch` como excepciones se quiera capturar y manejar de forma específica.

Cuando se **manejan varios tipos de excepciones** es relevante el orden en el que se capturan: deben ordenarse de más específicas a más generales.

La captura más genérica se implementa con un `catch` de la superclase `Exception`.

El método **`getMessage()`** se hereda desde `Throwable` y devuelve un `String` con el mensaje de error correspondiente. Este mensaje específico se pasa como parámetro al constructor de la excepción.

Es habitual utilizar `getMessage` al manejar una excepción para ofrecer mayor detalle al usuario.



2

Jerarquía de excepciones

02

Jerarquía

Throwable

- **Exception**

- ClassNotFoundException
- InterruptedException
- IOException
- SQLException
- FileNotFoundException
- **RuntimeException**
 - NullPointerException
 - ArithmeticException
 - ArrayIndexOutOfBoundsException
 - InputMismatchException

Checked: Verificadas en tiempo de compilación

Unchecked: Se dan en tiempo de ejecución

- **Error**

- StackOverflowError
- ThreadDead
- OutOfMemoryError
- VirtualMachineError



3

Tipos de excepciones

03

Tipos

Checked exceptions (verificadas):

- Heredan de Exception
- Se manejan declarándolas la firma del método (**throws**) o capturándolas explícitamente en un bloque **try-catch**
- Si no se manejan adecuadamente, **el código no compila**

03

Tipos

Unchecked exceptions (no verificadas):

- Heredan de RuntimeException
- **No es requerido manejarlas** explícitamente
- Son **errores de lógica** en tiempo de ejecución que se pueden corregir mejorando el código con un buen diseño o validaciones.

Errors (Errores)

- Errores graves que generalmente están fuera del control del programador y no deben manejarse explícitamente. Si se producen, debe detenerse la ejecución.
- Heredan de la clase Error



4

Excepciones
personalizadas

PROMETEO

04

Excepciones personalizadas

Permiten tratar **excepciones** específicas de una cierta aplicación, detallando situaciones inusuales en el **ámbito funcional** (lógica de negocio) de la aplicación.

Para crearlas, se define la clase del tipo de excepción personalizada heredando de Exception o RuntimeException

```
public class MiExcepcion extends Exception {  
    public MiExcepcion (String mensaje) {  
        super(mensaje); //constructor de Exception  
    }  
}
```


04

throw y throws

throw: se utiliza desde un método cuando se identifica una condición anómala para lanzar/elevar (acción) explícitamente una instancia de una excepción.

- En el momento de lanzar la excepción se crea el objeto con el constructor que recibe el mensaje que informa el detalle de la excepción.

throws: se utiliza en la firma (declaración) del método para indicar que dicho método puede propagar una o varias excepciones, **delegando su manejo al método invocador**

```
public void procesar() throws MiExcepcionPersonalizada {  
    throw new MiExcepcionPersonalizada("Error crítico");  
}
```



PROMETEO